Лабораторная работа 02

PicoRV32 на Tang Nano 9K: USB-UART, управление LED и синхронизация с учебной HTML-моделью

Методические указания и протокол фактически проведённого эксперимента
Редакция документа: 1.0 · **Экспериментальная система:** LAB-02 v0.1.0 · **Дата испытания:** 20 октября 2026 г.
Проект: Sipeed FPGA · **Плата:** Sipeed Tang Nano 9K · **FPGA:** GW1NR-9C
Разработка учебного эксперимента: Малачевский А. А. / A. A. Malachevsky, независимый исследователь · ORCID 0009-0008-6009-3196
Процессорное RTL-ядро: PicoRV32, Claire Xenia Wolf, ISC; используется исходный код с зафиксированной версией upstream.
Репозиторий: https://github.com/AIDevelopersMonster/Sipeed-FPGA

> **Подтверждённый результат.** На физической Tang Nano 9K реализован двусторонний USB-UART (COM10, 115200 8N1). Команды компьютера обрабатываются программой PicoRV32, изменяют MMIO-регистр светодиодов, а плата возвращает подтверждение. Учебная HTML-модель связывает переход через **шаг 14 из 19** с включением выбранного настоящего светодиода; движение назад выключает его, а повторный шаг вперёд включает снова. Проверены ручной и автоматический режимы. Статус: **MODEL_PASS · RTL_PASS · PNR_PASS · STA_PASS · BOARD_PASS · HTML_HARDWARE_PASS**.

Характер документа. Разделы 1–7 служат инструкцией студенту; раздел 8 — протокол проведённой работы, составленный по выводам команд и сообщениям о физическом наблюдении на испытательном стенде; раздел 9 — чистый бланк протокола. Данные, не зафиксированные в исходных результатах (например, SHA256 конечного `.fs` и номер платы), оставлены незаполненными, а не придуманы.

1. Цель, учебные результаты и маршруты

Цель: расширить ранее проверенную минимальную RISC-V SoC приёмником UART, выполнить управление физическими выходами FPGA через PicoRV32 и связать педагогическую модель исполнения инструкции SW с подтверждённым физическим эффектом.

По окончании работы студент должен уметь:

1. Различать процессор PicoRV32, программу RV32I в ROM, периферийные регистры MMIO и выводы FPGA.
2. Объяснять передачу команд по UART RX и обратное подтверждение по UART TX, отличие ответа L=n от электрического или оптического измерения.
3. Воспроизвести модель, RTL-тесты, Gowin PnR/STA и программирование **только конфигурационной SRAM**.
4. Проверить включение каждого выбранного LED, включая смену состояния при движении **13 → 14 → 13 → 14** в HTML.
5. Различать синхронизацию *события и подтверждения* и непосредственную тактовую трассировку реального процессора.

Маршрут A — демонстрационный: преподаватель предоставляет уже проверенную конфигурацию `.fs` вместе с SHA256; студент загружает её в SRAM, проверяет UART и работает с HTML. Готовый испытуемый `.fs` не следует предполагать в публичном Git-репозитории: файл сборки может быть исключён из Git.

Маршрут B — инженерный (выполненный в данном протоколе): студент самостоятельно запускает модель и RTL-тесты, собирает проект в Gowin, исследует STA, программирует SRAM, выполняет COM- и HTML-тесты. Все этапы выполняются на одной и той же экспериментальной версии LAB-02.

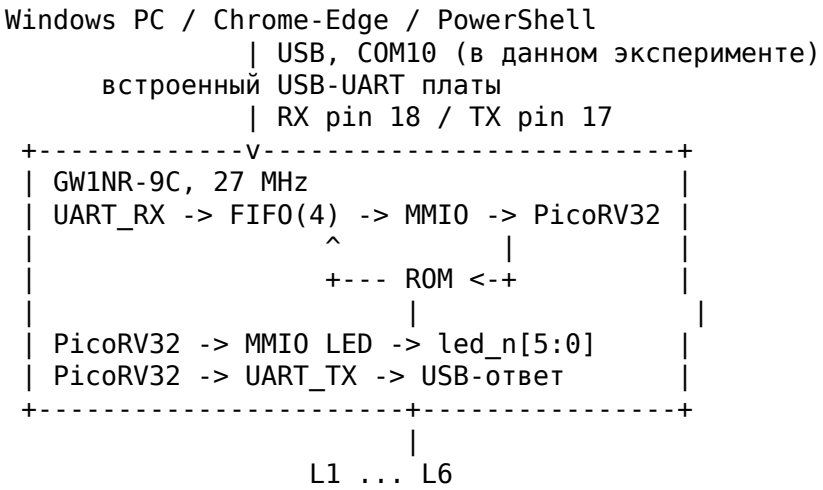
2. Оборудование и подготовка

Компонент	Фактически использовано / требуется
FPGA-плата	Sipeed Tang Nano 9K, кристалл семейства GW1NR-9C
Подключение	Один штатный USB-кабель с передачей данных; **внешний USB–TTL адаптер не нужен**
Частота SoC	Внешний генератор 27 МГц, период ограничения 37,037 нс
Последовательный интерфейс	Встроенный USB–UART платы; в испытании **COM10**, 115200 бод, 8N1
Компьютер	Windows 10, PowerShell 7, Git и Python 3
Проверка RTL	Icarus Verilog (`iverilog`, `vvp`)
FPGA toolchain	Gowin V1.9.9 Beta-4 Education, `gw_sh.exe`, Programmer CLI
Браузер	Chrome или Edge с Web Serial, доступный через `localhost`

Электрические назначения в проверенном файле ограничений: UART TX — FPGA pin 17, UART RX — pin 18; LED L1–L6 — pins **10, 11, 13, 14, 15, 16**. LED активны *низким* уровнем: логическая `1` в регистре `leds` означает `0` на соответствующем `led_n[i]` и включённый светодиод. Дополнительные провода для лабораторной не требуются.

Правила безопасности и воспроизводимости: работаем с правильным кристаллом и существующим USB-кабелем; не меняем напряжения выводов; закрываем другие терминалы до открытия COM; программируем **SRAM**, не `embFlash`. Сборка или успешный UART-тест не доказывают, что произвольная будущая версия SoC прошла проверку: любое изменение RTL/ROM требует соответствующего повторения тестов.

3. Архитектура и проверяемые свойства



Процессор исполняет **118 машинных инструкций RV32I** из 1-КиБ ROM (оставшиеся слова ROM заполняются NOP). Здесь нет RAM, стека, C runtime, прерываний или операционной системы. UART RX имеет двухкаскадный синхронизатор, выборку в середине бита и FIFO на четыре байта; предусмотрены признаки переполнения и ошибки стопового бита. Этот минимальный приёмник **не** является многократным передискретизатором с голосованием большинства. Запросы следует отправлять по одному с ожиданием ответа.

3.1. Карта памяти MMIO

Адрес	Регистр	Доступ / смысл
`0x20000000`	LED	R/W, младшие 6 бит
`0x20000004`	CYCLES	R/O, свободно работающий счётчик

Адрес	Регистр	Доступ / смысл
0x20000008	DEVICE_ID	R/O, 0x5A5A0001 (программная сигнатура, не JTAG ID)
0x2000000C	TEST_STATUS	R/W, сохранён из исходной SoC
0x20000010	UART_TX	запись младшего байта
0x20000014	UART_STATUS	бит 0 — TX готов
0x20000018	UART_RX_DATA	чтение извлекает байт из FIFO
0x2000001C	UART_RX_STATUS	бит 0 — данные есть; бит 1 — overrun; бит 2 — framing error; запись очищает флаги ошибок

3.2. Протокол и таблица светодиодов

Символ с ПК	Маска LED	Активный физический вывод	Ответ PicoRV32
0	0x00	все HIGH, LED выключены	L=0
1	0x01	pin 10 → LOW, L1	L=1
2	0x02	pin 11 → LOW, L2	L=2
3	0x04	pin 13 → LOW, L3	L=3
4	0x08	pin 14 → LOW, L4	L=4
5	0x10	pin 15 → LOW, L5	L=5
6	0x20	pin 16 → LOW, L6	L=6
?	не изменяет	чтение текущего MMIO-состояния	L=n

После сброса прошивка обнуляет LED и отправляет READY\n. Команда 0–6 вызывает **реальную запись SW в регистр LED**, затем чтение LW этого же регистра. При совпадении возвращается L=n\n, при несовпадении — F\n; неизвестные символы дают E\n (CR/LF игнорируются). **n — номер выбранного LED, а не двоичная маска.** Возврат L=n подтверждает *прочитанное значение MMIO*, но не заменяет визуальную проверку свечения.

4. Маршрут B: инженерная сборка и испытание

Выполняйте команды из указанного каталога. Существующую проверенную ветку main не изменять; LAB-02 собирается в feature/picorv32-uart-console-lab02. В рабочей копии могут находиться семь других неотслеживаемых файлов учебного комплекта — они **не относятся** к LAB-02 и не должны удаляться или добавляться массовой командой git add -A.

Шаг B1. Проверка исходников и сборочного маршрута

```
Set-Location C:\GitHub\Sipeed-FPGA
git branch --show-current
git status -sb
Test-Path .\01-PicoRV32\rtl\uart_rx_8n1.v
Select-String -Path .\01-PicoRV32\gowin_build.tcl -Pattern 'uart_rx_8n1.v'
```

Обязательная строка в 01-PicoRV32/gowin_build.tcl:

```
add_file rtl/uart_rx_8n1.v
```

В ходе данной работы сначала возникла ошибка Gowin EX3937: Instantiating unknown module 'uart_rx_8n1'. Причина: файл RX был включён в .gprj, но **не был включён в gowin_build.tcl, запускаемый CLI-скриптом**. После добавления строки сборка прошла. Это исправление необходимо сохранять в итоговом коммите: изменение только .gprj недостаточно.

Шаг B2. Модель прошивки и RTL-тесты

```
Set-Location C:\GitHub\Sipeed-FPGA
& .\01-PicoRV32\tools\test_lab02.ps1 -ModelOnly
& .\01-PicoRV32\tools\test_lab02.ps1
```

Ожидаемые сообщения: `ASSEMBLE PASS`, `LAB02_FIRMWARE_MODEL_PASS`, `LAB02_RX_RTL_PASS`, `UART TB PASS`, `LAB02_SOC_RTL_PASS`. Модель проверяет инструкции и протокол, RTL-тесты — приём/передачу UART и полную SoC. Это **не** аппаратное испытание.

Шаг B3. Gowin PnR и статический временной анализ

```
& .\01-PicoRV32\tools\build_gowin.ps1
```

Должен появиться `01-PicoRV32/impl/pnr/picorv32_nano9k.fs`. В отчёте `picorv32_nano9k.tr.html` проверьте `Actual Fmax`, setup/hold TNS и число нарушенных конечных точек. При 27 МГц требуется не просто успешный выход Gowin, а удовлетворение временных ограничений. Отдельно документируйте используемые ресурсы и расчётные условия STA.

Шаг B4. Программирование только SRAM

Закройте все COM-терминалы; используйте путь к **новому** `.fs`, а не произвольному старому файлу.

```
Set-Location C:\GitHub\Sipeed-FPGA\01-PicoRV32
$prog = 'C:\Gowin\Gowin_V1.9.9Beta-4_Education\Programmer\bin\programmer_cli.exe'
$fs = (Resolve-Path '.\impl\pnr\picorv32_nano9k.fs').Path
Get-Item $fs | Select-Object Name, Length, LastWriteTime
& $prog --device GW1NR-9C --operation_index 2 --fsFile $fs --cable-index 1
"PROGRAMMER_EXIT_CODE=$LASTEXITCODE"
```

Для испытанного комплекта `--operation_index 2` соответствует загрузке **временной конфигурации в SRAM**. Не использовать здесь операции записи embFlash. Зафиксируйте код возврата и SHA256 файла, если составляете самостоятельный протокол.

Шаг B5. Двусторонняя проверка UART и физический LED

```
Set-Location C:\GitHub\Sipeed-FPGA
& .\01-PicoRV32\tools\probe_lab02_serial.ps1 -PortName COM10
```

Скрипт быстро отправляет `0, 1, 4, ?, 6, 0` и проверяет ответы. При успехе выводит `LAB02_SERIAL_ACK_PASS`. **Не рассчитывайте успеть рассмотреть светодиоды:** этот тест автоматический. Для зрительного подтверждения передавайте команды по одной с паузой до Enter (пример ниже). COM-порт допускает только одного владельца; закройте PowerShell-тест перед Web Serial.

```
$sp = [System.IO.Ports.SerialPort]::new('COM10',115200,
  [System.IO.Ports.Parity]::None,8,[System.IO.Ports.StopBits]::One)
$sp.NewLine = "`n"
$sp.ReadTimeout = 2000
try {
  $sp.Open()
  Start-Sleep -Milliseconds 500
  $sp.DiscardInBuffer()
  foreach ($t in @(@('1','L1'),@('4','L4'),@('6','L6'),@('0','ALL OFF'))) {
    $sp.Write($t[0])
    $reply = $sp.ReadLine().Trim()
    Write-Host "$($t[0]) -> $reply; observe $($t[1])"
    Read-Host 'Нажмите Enter после осмотра платы' | Out-Null
  }
} finally { if($sp.IsOpen){$sp.Close()}; $sp.Dispose() }
```

Проверка глазами важна: она связывает ответ регистра с физическим результатом на плате. При ошибке первым делом проверить порт, конфигурацию LAB-02 в SRAM и отсутствие другого владельца COM; не начинать с переписывания Verilog или переустановки драйверов.

5. Связанная HTML-демонстрация — практикум

Запуск: один локальный HTTP-сервер, один браузер, тот же USB-кабель. Дополнительные USB–TTL, Wi-Fi, сервер в Интернете и прошивка Flash не нужны.

```
Set-Location C:\GitHub\Sipeed-FPGA
& .\01-PicoRV32\tools\launch_lab02_html.ps1 -Language RU
```

Откройте в Chrome или Edge:

`http://127.0.0.1:8765/PicoRV32_LED1_Linked_RU.html`

Для английской версии используйте `-Language EN` и файл `PicoRV32_LED1_Linked_EN.html`. `localhost` используется для работы Web Serial в подходящем защищённом контексте; COM10 выбирает сам пользователь в диалоге браузера. Перед подключением закройте обычный COM-терминал.

Задание H1. Выбор светодиода и первые 13 шагов

1. Нажмите **«Подключить COM»**, разрешите доступ к штатному UART платы.
2. Выберите любой LED, например **L4** (маска `0x08`, FPGA pin 14).
3. Перейдите с начального состояния по **шагам 1–13**, наблюдая ROM, регистры x5, x6 и поля `mem_addr`, `mem_wdata`, `mem_wstrb`, `mem_valid`, `mem_ready`.
4. До подтверждения записи в модельном шаге 14 выбранный светодиод должен оставаться выключенным.

Задание H2. Шаг 14 и обратимое учебное действие

1. **13 → 14.** HTML отправляет `'4'`; реальный PicoRV32 выполняет MMIO-запись и чтение, возвращает L=4; L4 на плате загорается, а HTML показывает подтверждённое состояние.
2. **14 → 13.** Кнопка «Назад» отправляет `'0'` и ожидает L=0; L4 гаснет.
3. **13 → 14.** Повторный шаг вперёд повторно включает L4; именно этот переход демонстрирует связь действия на экране с реальной периферией.
4. Пройдите оставшиеся шаги: в модели показывается бесконечный `JAL x0,0`, LED остаётся включённым.

Задание H3. Автоматический режим и другие LED

Запустите автоматическое прохождение от начального состояния; убедитесь, что аппаратное действие привязано к шагу 14, а не к произвольному времени. Повторите на нескольких выбранных светодиодах. По окончании нажмите **«Отключить»**, затем остановите локальный сервер `Ctrl+C`.

Принципиальная граница достоверности. В HTML показаны **4 примерные инструкции** `LUI`, `ADDI`, `SW`, `JAL` и **19 педагогических событий**. В настоящей FPGA работает другая, **118-инструкционная** программа командного UART-контроллера. Браузер не останавливает ядро на каждой инструкции; при движении назад процессор **не обращает своё прошлое**, а получает *новую команду* выключения LED. Синхронизировано событие изменения выхода и подтверждение L=n, **не тактовое исполнение** внутреннего PicoRV32.

6. Критерии сдачи и контрольные вопросы

Работа зачтена, если представлены: журнал модели/RTL (для маршрута B), путь и хеш построенного `.fs` (если файл имеется у исполнителя), STA без setup/hold нарушений, положительный UART ACK, визуальное включение/выключение хотя бы трёх LED и подтверждённый переход **13 → 14 → 13 → 14**. Статусы разных стадий фиксируются раздельно, без искусственного расширения области проверки.

Вопросы для защиты: Почему L=4 означает выбор L4, но `mem_wdata=8`? Что изменяется на физическом pin 14? Почему UART-ACK не является оптическим измерением? Зачем ждать ответа перед следующей командой? Почему положительный setup slack не подтверждает идеальную устойчивость асинхронного UART во всех условиях? Что означает движение назад в HTML по отношению к реальной программе CPU? Почему `.fs` и `boot.hex` — разные объекты?

7. Что не проверялось

Не проводились исчерпывающие тесты ISA RV32I, оценка надёжности UART при помехах/потере байтов или плотном потоке команд, проверка на других платах и версиях Gowin, измерение токов LED, оптическое измерение свечения, запись в `embFlash`, потактовая трассировка процессора или аппаратная остановка на каждой из 19 учебных фаз. В отчёте STA подтверждены ограниченные пути домена `clk27`; это не универсальная сертификация всех асинхронных интерфейсов.

8. Фактический протокол испытаний

Проведено: 20.10.2026; Windows / PowerShell 7; Sipeed Tang Nano 9K (GW1NR-9C), тактирование 27 МГц; USB-UART **COM10**, 115200 8N1; Gowin **V1.9.9 Beta-4 Education**. Серийный номер платы и SHA256 итогового битстрима в предоставленных выводах не зафиксированы.

Этап	Фактические наблюдения	Итог
Ассемблер	**118** инструкций; ROM **1024 байта**; **22** метки	PASS
Модель прошивки	**422** исполненных инструкций; READY, L=0, L=1, L=4, L=6, L=6, E, L=0; записи LED `[0,0,1,8,32,0]`	MODEL_PASS
UART RX RTL	Нормальные байты, `0x00`, ложный старт, ошибка кадра	RTL_PASS
UART TX RTL	Передача `0xA5` как UART 8N1	RTL_PASS
Полная SoC	Сообщения READY, команды `1,4,?,0`, ошибочная команда, физические сигналы `led_n` в симуляции	RTL_PASS
Gowin	Исправлено отсутствие `uart_rx_8n1.v` в `gowin_build.tcl`; затем полный PnR завершился	PNR_PASS
STA	`clk27` **27,000 МГц**; Actual Fmax **52,522 МГц**; Setup TNS **0**, Hold TNS **0**; 0 нарушенных endpoints	STA_PASS
Физический UART	`0→L=0`, `1→L=1`, `4→L=4`, `?→L=4`, `6→L=6`, `0→L=0`; вывод `LAB02_SERIAL_ACK_PASS`	BOARD_PASS (UART/MMIO)
Физические LED	При удерживаемых командах визуально подтверждены **L1, L4, L6** и состояние «все выключены»	BOARD_PASS (LED)
HTML + FPGA	Ручное прохождение 19 шагов, автоматический режим, шаг назад выключает, шаг вперёд включает; повторено с **тремя** выбранными LED (их номера отдельно не записывались)	HTML_HARDWARE_PASS

Ресурсы PnR: 1868 LUT/ALU/ROM16 = 1512 LUT + 356 ALU; 767/6693 регистров (~12%); 1127/4320 CLS (~27%); BSRAM **12%**.
Временной отчёт: 2154 пути, 1883 конечные точки; худший setup slack **+17,997 нс**, худший hold slack **+0,563 нс**. Условия: setup Slow 1,14 В / 85 °C; hold Fast 1,26 В / 0 °C.
Сформированный `.fs`: `picorv32_nano9k.fs`, **3 558 473 байта**, 20.10.2026 09:14:59 по времени файла.

Граница подтверждения: выводы команд приведены по протоколу испытания; зрительное наблюдение светодиодов и поведения HTML сообщено испытателем. Отдельный синхронный видеозапись/лог транзакций на уровне GPIO к материалам не приложен.

Итоговый вывод: поставленная функциональная задача выполнена. Система переводит действия студента в подтверждённые команды реальному процессору и воспроизводимые состояния физического LED. При этом образовательная последовательность из 19 шагов остаётся концептуальной моделью. Работу **LAB-02 v0.1.0** можно считать завершённой на заявленном уровне проверки; публикация изменений в GitHub является отдельной операцией и не утверждается данным протоколом.

9. Чистый бланк протокола студента

Поле	Заполнить
ФИО, группа / исполнитель	_____
Дата и время, ОС / PowerShell	_____
Плата, маркировка FPGA, номер экземпляра	_____
Ветка Git / идентификатор коммита	_____
Gowin / Icarus / Python (версии)	_____
COM-порт, параметры UART	_____
`.fs` (имя, размер, SHA256)	_____
MODEL / RTL / PNR / STA (показатели)	_____
UART: команды → ответы	_____
LED: выбранные L1–L6 и наблюдение	_____
HTML: шаг 13 → 14 → 13 → 14	_____
Автоматический режим / дополнительные LED	_____
Ошибки, ограничения, самостоятельный вывод	_____

Материалы и ссылки

- Исходники PicoRV32 и SoC; программа RV32I; RTL UART RX.
- RTL-тест LAB-02; проверка физического UART; запуск HTML.
- HTML (RU) · HTML (EN).
- Репозиторий проекта · Оригинальное ядро PicoRV32.